



Introduction

▼ Type	 Lecture
📅 Date	@December 27, 2021
☰ Lecture #	1
🔗 Lecture URL	https://youtu.be/3a1FXBR6QXY
🔗 Notion URL	https://21f1003586.notion.site/Introduction-95ec26f2dc934c96a69762cad8582900
# Week #	1

Programming Languages

- A language is a medium for communication
- Programming languages communicate computational instructions
- Originally, directly connected to architecture
 - Memory locations store values, registers allow arithmetic
 - Load a value from memory location M into register R
 - Add the contents of the register R_1 and R_2 and store the result back in R_1
 - Write the value in R_1 to memory location M'
- Tedious and Error prone process

Abstraction

- Abstractions used in computational thinking
 - Assigning values to named variables
 - Conditional execution
 - Iteration
 - Functions/Procedures, recursion
 - Aggregate data structures — arrays, list, dictionaries
- Express such ideas in the programming language
 - Translate “high level” programming language to “low level” machine language
 - Compilers, interpreters

- Trade off expressiveness for efficiency
 - Less control over how code is mapped to the architecture
 - But fewer errors due to mismatch between intent and implementation

Styles of programming

- Imperative vs Declarative
- Imperative
 - How to compute
 - Step-by-step instructions on what is to be done
- Declarative
 - What the computation should produce
 - Often exploit inductive structures, express in terms of smaller computations
 - Typically avoid using intermediate variables
 - Combination of small transformations — functional programming

Imperative vs Declarative Programming, by example

- Add values in a list
- **Imperative** (in Python)

```
def sum_list(l):
    sum = 0
    for x in l:
        sum += x
    return sum
```

- Intermediate values `sum, x`
- Explicit iteration to examine each element in the list
- **Declarative** (in Python)

```
def sum_list(l):
    if l == []:
        return 0
    else:
        return l[0] + sum_list(l[1:])
```

- Describe the desired output by induction
 - Base case → Empty list has sum 0
 - Inductive step → Add the first element to the sum of the rest of the list
- No intermediate variables

-
- Sum of squares of even numbers upto `n`
 - **Imperative** (in Python)

```
def sum_square_even(n):
    sum = 0
    for x in range(n + 1):
        if x % 2 == 0:
            sum += x * x
    return sum
```

- We can code functionally in an imperative language
- Helps us identify natural units of (reusable) code
- **Declarative** (in Python)

```
def even(x):
    return x % 2 == 0

def square(x):
    return x * x

def sum_square_even(n):
    return sum(map(square, filter(even, range(n + 1))))
```

Names, types and values

- Internally, everything is stored as a sequence of bits
- No difference between data and instructions, let alone numbers, characters, booleans
 - For a compiler or interpreter, our code is its data
- We impose a notion of type to create some discipline
 - Interpret bit strings as "high level" concepts
 - Nature and range of allowed values
 - Operations that are permitted on these values
- Strict type-checking helps catch bugs early
 - Incorrect expression evaluation — like dimension mismatch in science
 - Incorrect assignment — expression value does not match variable type

Abstract datatypes, object-oriented programming

- Collections are important
 - Arrays, lists, dictionaries
- Abstract data types
 - Structured collection with fixed interface
 - Stack, for example, is a sequence but only allows `push` and `pop`
 - Separate implementation from interface
 - Priority queue allows `insert` and `delete-max`
 - Can implement a priority queues using sorted or unsorted lists, or using a heap
- Object-Oriented Programming
 - Focus on data types
 - Functions are invoked through the object rather than passing data to the functions
 - In python, `my_list.sort()` vs `sorted(my_list)`

What is yet to come ...

- Explore concepts in programming languages
 - Object-oriented programming
 - Exception handling, concurrency, event-driven programming
- Use Java as the illustrative language
 - Imperative, object-oriented
 - Incorporates almost all the features
- Discuss design decisions where relevant
 - Every language makes some compromises
- Understand and appreciate why there is a zoo of programming languages out there, *lol*
- And why new ones are still being created